Multi-Agent Chatbot for Efficient Interaction with Blockchain APIs

Sy-Hong-Duc Nguyen*[0000−1111−2222−3333], Tuan-Dat Trinh*†[0000−0002−3014−3737], and Quoc-Viet-Quang Tran[2222−−3333−4444−5555]

Hanoi University of Science and Technology, Hanoi, Vietnam
ducnsh.hust@gmail.com
dattt@soict.hust.edu.vn
flash291101@gmail.com

Abstract. This paper addresses the challenge of enabling end users to interact directly with various APIs that provide real-time data and perform actions, eliminating the need for programming or developing intermediate applications. Direct interaction with APIs offers significant benefits, including increased accessibility, reduced development time, and enhanced flexibility in meeting diverse user needs. We propose a chatbot solution that leverages Large Language Models (LLMs) to facilitate these interactions. The advanced language understanding and reasoning capabilities of LLMs underpin our approach, addressing challenges such as precise query refinement, planning the selection of one or more APIs, and extracting parameters from user queries for API input. Our novel architecture integrates question refinement, entity recognition, and API filtering modules, supported by a multi-agent chatbot system that plans and evaluates API usage. This multi-agent system operates as a collaborative team of specialized experts, iteratively handling complex queries. Experimental results demonstrate that our system achieves a 91.95% accuracy rate with minimal response time. This approach simplifies the development of chatbots across various domains by leveraging available APIs, making it easier to build sophisticated, context-aware systems.

Keywords: Chatbot · Multi-Agent System · Blockchain · LLM · Large Language Model

1 Introduction

The rapid evolution of artificial intelligence (AI) has led to the proliferation of conversational agents, commonly known as chatbots, that leverage Large Language Models (LLMs) to interact with users in natural language. Traditionally, these chatbots operate based solely on the knowledge embedded in their training data, which limits their ability to access real-time information or data beyond their training cut-off. This constraint significantly reduces the effectiveness of LLM-based chatbots in dynamic and data-intensive environments, such as blockchain technology, where real-time data is critical.

To address this challenge, we propose a novel chatbot system that extends the functionality of LLMs by integrating real-time data retrieval through a suite of Application Programming Interfaces (APIs) within the blockchain domain. This development is particularly useful as blockchain ecosystems are complex and continuously evolving, requiring users to access up-to-the-minute data for informed decision-making. Furthermore, the

* The two authors contributed equally to this work
† Corresponding author

system is designed to handle both data-retrieval APIs and transaction-execution APIs, enabling a full range of interactions within the blockchain space.

The motivation behind this work stems from the growing demand for accessible tools that bridge the gap between technical complexity and user accessibility. Typically, interacting with APIs requires specialized knowledge and coding skills, which are barriers for non-developers. By embedding API integration into a chatbot, our system democratizes access to blockchain data, enabling end-users to query APIs using natural language without needing to understand the underlying technicalities. Although our system is demonstrated in the blockchain domain, it is designed to work with any domain that provides APIs for data access. The approach allows enterprises to maximize data utilization and API infrastructure, enhancing both user experience and operational efficiency across API-driven environments.

The proposed chatbot architecture addresses the challenges of integrating APIs into a chatbot system with a variety of AI models. First, the LLM-based Query Refinement Agent enhances user queries to improve precision by refining vague or ambiguous requests. Next, the Entity Recognition Module uses GLiNER model and Elastic Search to extract relevant entities from the user input, mapping them to API parameters. The API Filtering Modules, powered by LLM Embedder, an Bi-Encoder model, and BGE Rerank, a Cross-Encoder model, determines relevant APIs to apply to a given query, reducing unnecessary overhead and streamlining the execution process.

We designs a multi-agent system, consists of Planning, Validation, Evaluation, and Response Generation Agents. This system functions like a team of experts, collaborating in an iterative loop to plan, validate, and generate the final response. The Planning formulates the optimal strategy for handling user queries, the Validation remove hallucination when planning, the Evaluation checks the correctness of the plan, and the Response Generation compiles the final answer.

This novel architecture bridges the gap between LLM-powered chatbots and API-driven interactions, ensuring accurate and efficient communication with real-time data sources. Experimental results show that the system achieves 91.95% accuracy with minimal response times, making it highly suitable for dynamic environments like blockchain technology.

The rest of this paper is organized as follows: Section 2 introduces key terminology, followed by Section 3, which outlines the chatbot requirements. Section 4 reviews related work; Section 5 details the chatbot architecture. Section 6 discusses the AI models used for each module. Section 7 presents the experiments and results, and Section 8 concludes with insights and future research directions.

2 Terminology

Tool. In the context of this paper, a *tool* refers to a technique designed to address the limitations of LLM chatbots. Typically, an LLM chatbot can answer queries based on its pre-trained data and provided prompts. To handle real-time or specialized queries, such as those requiring external data, developers integrate *tools* into the chatbot system.

To develop a *tool*, a chatbot developer creates a function with clearly defined intent description and input parameters. For example, a weather forecasting *tool* may include a function that accepts location and time as parameters and calls an external API, such as Yahoo Weather, to fetch and return weather information. This function can also perform various operations, potentially involving multiple APIs, to achieve the desired outcome.

Upon receiving a user query, the chatbot application interacts with LLMs, passing the query along with relevant conversation history and prompts. Additionally, the query includes a prompt that details the available *tools*, instructing the LLM to utilize these *tools* if necessary. For example, the prompt might state: "*You have the following tools:*

{Detailed description of each tool}. If needed, suggest using these tools to answer the user's question and return the response in JSON format with tool name and parameters."

The decision to use a *tool* is made by the LLM. If the LLM determines that a *tool* is not required, it directly provides the response to the user. If a *tool* is required, the LLM returns a JSON string specifying which *tool* to use and its corresponding parameters. This JSON string is not sent to the end user; instead, the developer uses it to call the defined function. The results from this function are then processed and refined by the LLM to provide a comprehensive and accurate answer to the user.

In summary, a *tool* enhances the LLM's capabilities by enabling it to handle a broader range of queries through external data sources or complex operations. The flexibility and extensibility of these *tools* allow developers to create numerous functionalities tailored to specific needs.

Agent. In artificial intelligence (AI), an *agent* is a system or model capable of interacting with its environment to perform tasks, learn from feedback, and make autonomous decisions [5]. LLM Agents [14] are agents built on LLMs, such as GPT (Generative Pretrained Transformer). Unlike traditional agents, which follow predefined rules or rely on simple machine learning models for specific tasks, LLM Agents can handle more complex tasks by leveraging their ability to reason, understand natural language, plan, use tools, and manage memory. This enables them to adapt to a wider range of applications, especially in diverse and unpredictable scenarios.

LLM Agent systems can be categorized into two types: single-agent and multi-agent systems. A single-agent system uses one LLM agent that operates independently to complete tasks without requiring coordination with other agents. In contrast, a multi-agent system involves multiple LLM agents that interact and collaborate to solve more complex tasks that require coordination and information sharing among agents.

3 Chatbot Requirements

The objective is to develop a blockchain chatbot capable of providing information and executing various blockchain transactions, including those related to lending pools, staking, and governance. To achieve this, the chatbot must integrate multiple tools while leveraging the inherent knowledge of a large language model (LLM) to ensure accuracy and efficiency in both responses and actions.

The system comprises 26 tools[1], each corresponding to a specific API. The first tool introduces users to the features, functions, and operational guidance of the chatbot. The second tool retrieves information from the internet to address queries that neither the LLM nor other tools can answer, extending the chatbot's ability to provide up-to-date and detailed information from online sources. The remaining 24 tools are divided into two categories: informational tools and transactional tools.

Seven tools are designated for providing information, allowing the chatbot to answer user queries on the following topics: (1) token prices, (2) trending tokens, (3) wallet information, (4) borrow rates and deposit rates within lending pools, (5) the latest blockchain news, (6) rankings of DeFi (Decentralized Finance) applications like lending pools, DEXs (Decentralized Exchanges), and yield platforms, and (7) detailed information about lending pools, including total value locked (TVL), number of users, and number of borrows.

The remaining 17 tools handle transactions across lending pools, staking, and governance. Lending pool transactions include: (1) deposit tokens, (2) borrow tokens, (3) withdraw tokens, (4) repay tokens, (5) transfer tokens, (6) claim rewards, and (7) convert rewards. Staking-related transactions involve: (8) stake tokens, (9) withdraw from staking, (10) claim staking rewards, and (11) transfer staked tokens. Lastly, governance transactions cover: (12) create locks for asset management, (13) increase the quantity of

[1] `https://github.com/Duc-NSH/blockchain-chatbot-question-data`

locked assets, (14) extend unlock time, (15) withdraw assets from governance, (16) merge multiple locks, and (17) compound transactions for reinvestment.

The transactional tools gather essential details for each transaction, including the target contract address, function call, and parameters. After preparing this information, the tools generate a structured JSON containing all details needed to build an integrated user interface within the chatbot. Similar to other blockchain interactive interfaces, this setup allows users to review and confirm transaction parameters before execution, enhancing transparency and control. Final transaction approval is then managed by trusted crypto wallets like MetaMask, which authorize and execute the confirmed transactions with DeFi applications.

4 Related Work

Recent developments in LLMs have led to significant progress in enhancing chatbot capabilities, particularly in addressing the challenge of handling queries that fall outside the LLM's pre-trained knowledge or require interaction with external environments. This section discusses key approaches, including agent-based systems, tool learning, and frameworks like LangChain, that aim to extend the functionality of LLMs in real-world applications.

Yao et al. [18] introduced ReAct, a single-agent system that combines reasoning and action in LLMs. The system follows a continuous loop of three steps: reason, act, and observe, enabling the agent to update its actions based on new information from the environment. Although ReAct requires significant computational resources and struggles with long-term planning for complex tasks, it has laid important groundwork for future agent-based systems.

Zhang et al. [20] addressed some inherent limitations of LLMs, such as their inability to update knowledge, manage long-term memory, and perform actions autonomously. They proposed LLM-Embedder, a lightweight bi-encoder model that helps LLMs retrieve knowledge from various sources, including knowledge bases, tool repositories, and memory stores. This unified retrieval approach enhances LLMs' performance across diverse and complex scenarios, providing a more efficient solution than relying on separate embedding models for different tasks.

Another promising approach to overcoming the inherent knowledge limitations of LLMs is Retrieval-Augmented Generation (RAG) [4]. This method enhances LLM output by providing additional external data sources for reference before generating a response. Tool learning [12, 16] is a specific implementation of RAG, offering several advantages: (i) scalability and flexibility, as tools are modular and can be adjusted without affecting the overall system, (ii) task specialization, allowing developers to create tools tailored to specific queries, and (iii) accuracy enhancement by enabling LLMs to incorporate real-time or domain-specific data through tools, thus reducing hallucinations and outdated information. However, tool learning also introduces challenges, including increased system complexity and computational costs when integrating multiple tools.

Several frameworks have been developed to support the creation of LLM-based chatbots, each offering unique features and capabilities. LangChain[2], an open-source platform, enables the rapid development and integration of LLM applications by connecting them to external data sources and managing agents. It is particularly useful for quickly deploying simple applications but may face challenges in scaling and customization for more demanding tasks. Llama Index[3] is another framework that focuses on structuring and indexing data to make LLMs more efficient in querying and retrieving information, enhancing the model's ability to work with large datasets. Autogen[4], on the other hand,

[2] `https://www.langchain.com/`
[3] `https://www.llamaindex.ai/`
[4] `https://microsoft.github.io/autogen/`

provides a more robust framework for automating agent interactions. It enables the development of complex multi-agent systems where LLMs can coordinate tasks and share information seamlessly. These frameworks play a key role in extending the functionality of LLMs and improving the efficiency of chatbot applications, though each comes with its own set of trade-offs in terms of performance, flexibility, and scalability.

These advancements highlight the ongoing efforts to overcome the constraints of LLMs and expand their practical applications, especially in complex and dynamic environments.

5 Chatbot Architecture

The chatbot architecture, shown in Figure 1, comprises ten modules. When a user submits a query, the *Query Refinement* (Module 1), an LLM-based agent, refines the query for clarity. It leverages the user's conversation history and examples of query refinements, both stored as embedded vectors, along with entities extracted from the whole conversation. These elements are incorporated into the LLM's prompt, enabling the agent to generate a self-contained and clearer query that accurately reflects the user's intent without requiring further reference to the conversation history.

The refined query is subsequently processed by Modules 2, 4, 6, 7, and 9. Initially, Module 2 uses a lightweight entity recognition model to extract text segments as potential entities and classify them by type.The outputs from Module 2 are then directed to Module 3, which leverages *Elastic Search* to accurately identify entities and retrieve their corresponding details. This approach effectively handles spelling errors; for example, Module 2 might detect "Bitcon" as a potential entity, and Elastic Search would correct it to "Bitcoin". These two modules enhance the LLM's capabilities, as LLMs are inherently constrained to recognizing entities within their training data, and expanding their knowledge through retraining is resource-intensive. Accurate entity recognition are crucial for understanding the query correctly and selecting appropriate inputs for tools to provide precise responses to users.

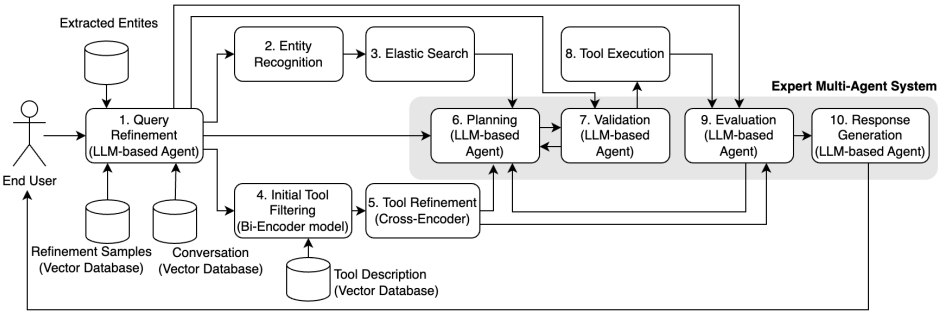

Fig. 1: Chatbot Architecture

To meet the diverse needs of users, the chatbot system integrates multiple tools (see Section 3). These tools are used by feeding their descriptions and input parameters into the LLM prompt. Including all available tools in the prompt can reduce efficiency and increase costs due to the excessive information processing. To address this, we implement a filtering process for the tools using two models: *Bi-Encoder* (Module 4) and *Cross-Encoder* (Module 5) [2]. The *Bi-Encoder* model first computes and stores embeddings in a vector database for all tool descriptions. The embedding of user's query then is compared with every stored embeddings to identify which tools are most likely to meet the current query's needs. Once the *Bi-Encoder* selects potential tools, the *Cross-Encoder* re-ranks them. Unlike the *Bi-Encoder*, the *Cross-Encoder* processes the input as pairs of the user's query and tool descriptions. It outputs a similarity score ranging from 0 to 1, indicating

the degree of relevance between the query and each tool description. The *Cross-Encoder* excels at capturing the relationship between the query and tool descriptions, resulting in higher accuracy compared to the *Bi-Encoder*. However, it is slower because it does not precompute and store embeddings for tool descriptions as the *Bi-Encoder* does.

Next, Modules 6, 7, 9, and 10 work together as a multi-agent system, with Modules 6 and 9 serving as the core agents. Module 6, the *Planning*, is an LLM-based agent that takes three inputs: (1) the refined query, (2) the identified entities and their detailed information, and (3) the list of tools filtered by the *Bi-Encoder* and *Cross-Encoder*. The *Planning* is responsible for devising a plan to use the tools, including the order of execution and the detailed input parameters for each tool. If the *Planning* determines that the available tools are unsuitable for the query, it will not generate a plan. In such cases, Module 7, the *Validation*, assesses the *Planner*'s decision to prevent errors or hallucinations. If the plan is deemed viable, it is passed to Module 8 for execution.

Module 8, the *Tool Execution*, implements the plan. Its results are then passed to Module 9, the *Evaluation*, another LLM-based agent. The *Evaluation* receives the tools, the refined query, and the execution results from the *Planning* to assess whether the plan meets the user's needs. If the plan is insufficient and can be improved using available tools, the *Evaluation* develops additional plans based on prior ones to optimize the outcome, and the process loops back to the *Planning*.

If the plan meets the requirements, the *Evaluation* notifies Module 10, the *Response Generation*, to compile and deliver the final response to the user. In cases where the plan is not fully satisfactory and no further tools can improve the result, the *Evaluation* still notifies the *Response Generation*. This module will then summarize what has been resolved and acknowledge the unresolved issues to the end user.

The following section details the *Question Refinement, Entity Recognition, Tool Filtering, Planning, Validation, Evaluation*, and *Response Generation* modules.

6 AI Models for Architectural Modules

6.1 Question Refinement Agent

In a question-answering system, accurately understanding user queries is crucial. However, users often phrase questions incompletely or refer to previous questions, which can lead to misunderstandings of their intent. For instance, consider the sequence: *"(1) What are the top ranking tokens now? (2) What is the current price of the third one? (3) How about its deposit rate on Venus lending pool?"*. Only the first query is complete and clear; the subsequent queries lack necessary information and depend on context from previous questions. Expecting users to provide all required details in every individual query is unrealistic and unnatural.

To address this issue, we rewrite queries into clear, standalone questions based on conversational context [3]. This method enhances response accuracy by refining the query into a more explicit form and reduces computational costs by processing only the refined query rather than the entire conversation history.

The question refinement process employs two vector databases. First, the conversation history between the user and the system is stored as "Question-Answer" pairs, which are then vectorized and stored in the *Conversation Vector Database* using Qdrant[5]. Additionally, examples with the format "conversation history – original query – refined query" are created and stored in the *Refinement Sample Vector Database* to guide the LLM in refining queries more accurately.

The LLM agent for query refinement utilizes three types of input data. (1) First, it identifies relevant segments of the conversation history by vectorizing the user's query and matching it with the *Conversation History Vector Database*. This approach avoids

[5] `https://qdrant.tech/`

processing all previous interactions. (2) It selects query refinement examples that are contextually relevant from the *Refinement Sample Vector Database*. (3) It incorporates blockchain entities extracted from prior queries. These three inputs are combined in the prompts, allowing the LLM to refine queries effectively by leveraging its reasoning and language understanding capabilities.

We employ the LLM-Embedder model [20], with 109 million parameters, to encode text into vector space. This bi-encoder model is versatile, capable of handling tasks such as *knowledge retrieval*, *memory augmentation*, *in-context learning through example retrieval*, and *tool selection* by leveraging instruction-based fine-tuning technique.

These tasks are highly relevant to our chatbot system. *Memory augmentation* retrieves relevant conversation history, used in the question refinement agent, while *in-context learning* allows the retrieval of similar examples for query refinement; *tool selection* is applied to identify the appropriate tools to answer user queries (see Section 6.3). Due to the challenges of data generation and the strong performance of the base model, we did not fine-tune the LLM-Embedder for the first two tasks. Fine-tuning was applied solely for the tool selection task, details of which are presented in Section 6.3.

Each task is assigned a specific instruction, consisting of a **Query** and a **Key** pair, to guide the LLM-Embedder in adjusting vector representations according to the task requirements [1, 20, 15]. The **Query** instruction is applied to user queries, and the **Key** instruction is used for embedding information stored in the vector database. For example, in the task of finding relevant examples for query refinement, the **Query** instruction is: "*Convert the following dialogue into vector to find useful examples: User Question*" while the **Key** instruction is: "*Convert this example into vector for retrieval: Example*".

6.2 Entity Recognition with GLiNER and Elastic Search

Accurate recognition of blockchain entities (e.g., tokens, projects, chains) is essential for our chatbot architecture, as it ensures correct understanding of user queries and appropriate tool selection. However, entity recognition encounters several challenges: (i) the extensive variety of entities, (ii) the continual introduction of new entities, (iii) the interchangeability of names and symbols, and (iv) the absence of standardized naming conventions, leading to potential name overlaps. For example, the token "Bitcoin" has the symbol "BTC", while "blackrocktradingcurrency" also uses "BTC." Thus, queries like "What is the current BTC price?" necessitate accurate differentiation between these entities.

Our approach to entity recognition involves two steps: extracting potential entity strings from user queries using the GLiNER model [19], and employing Elastic Search with BM25, Fuzzy Search, Prefix, and Wildcard algorithms to find the closest matching entity from our continuously updated and expanded entity database. This method offers several advantages: (i) GLiNER, a lightweight language model with approximately 400 million parameters, ensures high efficiency in entity recognition, and (ii) the entity database is continuously updated independently of GLiNER, enabling recognition of future entities without retraining and effectively handling misspelled entities.

GLiNER [19] is designed for Named Entity Recognition (NER) and presents notable improvements over traditional NER systems and LLMs. Unlike traditional NER models restricted to fixed entity types, GLiNER can recognize various entity types using natural language prompts. This flexibility allows it to adapt to different domains without extensive retraining. To enhance the accuracy of entity recognition, we fine-tuned GLiNER.

Training data was generated using GPT-4 Turbo through the following process: (i) generating text segments containing blockchain-related entities and topics, annotated with XML syntax to mark entity positions, (ii) diversifying the entities and topics to enrich the data, and (iii) augmenting the data by altering entity names within XML tags, creating

multiple variants with the same grammatical structure to enhance the model's ability to recognize entities based on sentence patterns rather than specific names.

The resulting training dataset comprises 10,000 samples of text containing blockchain-related entities. The core model, deBERTa-v3, is a bidirectional language model with high performance in NER tasks. GLiNER's non-pretrained layers have a width of 768 and a dropout rate of 0.4. Training utilized the AdamW optimizer, with a base learning rate of 1e-5 for the backbone transformer and 5e-5 for non-pretrained layers. Negative Entity Sampling with a ratio of 0.5 was employed to help the model distinguish between genuine entities and non-entities.

6.3 Tool Filtering with Bi-Encoder and Cross-Encoder Models

Incorporating tools is essential for bridging LLMs with real-time data. For example, a tool that accepts a token name and returns its price allows the LLM to answer questions like *"What is the current price of Bitcoin?"*. A chatbot equipped with multiple tools can address diverse user requests. However, integrating too many tool descriptions into the LLM increases context length and complexity, leading to the **lost in the middle** issue [10], where the LLM tends to forget critical information midway through, making it difficult to accurately select the right tool.

To address this, irrelevant tools must be filtered out before passing their description to the LLM. The tool filtering process involves two stages. First, the refined query (see Section 6.1) is vectorized using LLM Embedder [20], a Bi-Encoder model, and matched with pre-embedded tool descriptions stored in a vector database. This initial step filters out the most relevant tools. Next, these candidate tools are re-ranked by the BGE Rerank[6], a Cross-Encoder model, which assigns scores to find the tools most appropriate for the query. This two-stage filtering process improves scalability, reduces computational cost, and enhances accuracy [2].

To fine-tune both the LLM-Embedder and BGE Rerank models for tool filtering, we generated a training dataset of 70,000 samples using GPT-4o in the blockchain domain. The data generation process includes three steps: (1) GPT-4o first creates a list of major blockchain topics, such as DeFi, NFTs, wallets, and security, (2) each major topic is expanded into subtopics, detailing specific aspects of the field, and (3) detailed tool descriptions for each subtopic are generated and stored in the training samples.

Each training data sample consists of three fields: the user query, the tools needed to handle the query (stored as tool positives or pos), and the associated topic. The queries are designed to simulate real-world scenarios, ranging from straightforward ones that directly match the tool description to more complex ones requiring reasoning and coordination across multiple tools. This diversity enables the LLM-Embedder and BGE Rerank models to learn optimal tool selection, ensuring both scalability and generalization. The queries fall into three categories: (1) those requiring a single tool, (2) those involving multiple tools from the same subtopic, and (3) those requiring multiple tools from different subtopics or broader topics.

Negative samples (neg) are included in the training process to help the model distinguish between relevant and irrelevant tools. These negatives are generated using methods such as BM25 [12] or by selecting tools from unrelated topics compared to the positive set. This enables the model to optimize its filtering capability, accurately selecting tools for each query.

During the fine-tuning process for LLM-Embedder and BGE Rerank, task-specific instructions were integrated into the training data. A small learning rate (1e-5) was used to preserve the models' pre-existing knowledge. Fine-tuning was performed over four epochs with a cross-entropy loss function [11], with the goal of ranking the most suitable tools highly while discarding irrelevant ones.

[6] `https://huggingface.co/BAAI/bge-reranker-base`

6.4 Expert Multi-Agent System

In complex chatbot applications where LLMs must answer questions not directly covered by their training data, Retrieval-Augmented Generation (RAG) models [4] can be employed to enrich responses using external information. However, RAG struggles with intricate queries requiring multistep reasoning, planning, and the use of external tools. To overcome these challenges, an agentic RAG architecture [8] is used. This system blends RAG with agent-based decision-making, enabling dynamic coordination between LLMs and various tools. Agentic RAG not only augments prompts with relevant external data but also orchestrates a series of actions, selecting the most suitable models and tools for each task. It functions like a team of experts, each with specialized skills and knowledge, working together to efficiently tackle complex information requests.

Using the agentic RAG model, we developed a multi-agent system comprising four primary agents: the *Planning*, the *Validation*, the *Evaluation*, and the *Response Generation* agents. This system is implemented as a feedback loop algorithm that continuously refines its search for information to address user queries. The algorithm terminates either when the maximum number of iterations is reached or when the final agent receives sufficient and accurate information to provide a response.

Our multi-agent system builds on concepts similar to Reflexion [13] and CRITIC [6] by incorporating a closed-loop feedback mechanism among its agents. It also integrates strengths from the LLM Compiler [9], an agent designed for parallel task planning and execution. The advantages of the system include: (i) no requirement for model fine-tuning or weight updates, (ii) the ability to learn from mistakes through iterative feedback and adjustments, and (iii) generation of concise and relevant answers to user queries.

6.4.1 Planning Agent The *Planning Agent* is responsible for strategizing the use of necessary tools to obtain external information for answering queries. It can be activated up to n times, corresponding to n iterations in the algorithm. In each iteration, the agent refines its strategy based on errors identified in previous attempts.

The output of the *Planning Agent* is in JSON format, containing all parameters required for the *Tool Executor* module (see Section 5) to execute the tools. To ensure accuracy and consistency in the output, the model used for this agent operates with a temperature setting of 0. This setting controls the level of creativity in the model: a lower temperature results in more focused and consistent outputs, while a higher temperature would produce more varied results. Our *Planning Agent* is inspired by the ReAct approach [18], combining reasoning and action, and enhancing it with parallel multi-plan execution. Each planning step includes: (i) the step number, starting from 1, (ii) reasoning and objectives for the step, (iii) the action of selecting the appropriate tool, (iv) the input parameters for the tool, which require information of recognized entity in the user query, and (v) identifying any dependencies of this step on others.

To leverage the full reasoning capability of LLMs, the *Planning Agent* may opt not to generate a plan if the provided tools are deemed unnecessary or unsuitable for the user query. The output from the *Planning Agent* is passed through the *Validation Agent*.

6.4.2 Validation Agent The *Validation Agent* addresses the issue of hallucination [7], a common phenomenon in LLMs where the generated information may appear plausible but is actually incorrect or fabricated. Hallucination is more likely to occur when the model is queried about real-world data or information beyond its training.

The primary function of the *Validation Agent* is to verify the accuracy of the responses generated by the *Planning Agent*. It evaluates whether the *Planning*'s decision to not generate a plan is justified or a result of hallucination. If the decision is deemed unreliable, the *Validation Agent* will request a revised plan from the *Planning Agent*. Conversely, it will pass the plan to the *Tool Executor* module for execution.

The *Validation Agent* takes two inputs: the user query and the *Planning Agent's* response. For instance, if a user asks, *"What is the current price of BTC?"* and the *Planning*

Agent provides an immediate response such as *"BTC price is $61,328"* without a plan, the *Validation Agent* will require the *Planning Agent* to revise the response due to the lack of supporting evidence.

6.4.3 Evaluation Agent The *Evaluation Agent* assesses the effectiveness of the planning phase, as the *Planning Agent* have developed plans without actual tool execution results. The *Evaluation Agent* determines whether the plan adequately addresses the user query. If the plan is insufficient and additional tools are available, the agent will create further planning steps. If the plan meets the requirements, the *Evaluation Agent* concludes the planning phase and forwards the results to the *Response Generation Agent*.

6.4.4 Response Generation Agent The *Response Generation Agent* is the final component in our multi-agent system, responsible for synthesizing information and generating the most natural response for the user. To ensure that the output is both natural and diverse, we employ the Chain-of-Thought prompting technique [17] and set the temperature parameter to its maximum value of 1 for the model used by this agent. This approach facilitates the generation of coherent and contextually appropriate responses.

7 Experiments

7.1 Experiment Setup

We evaluated the chatbot using a dataset of 298 diverse questions[7], organized into conversational dialogues. Each dialogue contained 5 to 6 questions, where later questions could require context from the preceding conversation to be understood correctly. The questions varied in complexity: some could be answered using a single tool, while others required multiple tools. They ranged from straightforward queries closely aligned with the intended tools, demanding minimal reasoning, to more complex ones that required extensive reasoning and contextual inference due to mismatches between the questions and available tools.

To validate the chatbot's real-time information accuracy, we conducted a manual evaluation with human evaluators, including NLP-experienced developers and blockchain advisors, as automatic metrics like BLEU and ROUGE measure only textual similarity without verifying accuracy. Evaluators assessed each response for relevance and correctness, specifically verifying whether each answer aligned with the question and if the data matched results from relevant APIs.

We used four evaluation metrics: (i) **Accuracy Rate**: The proportion of correct answers out of the total number of questions. An answer is deemed correct if it provides complete and accurate information as verified by human evaluators. (ii) **Response Time**: The average time taken by the chatbot to completely answer each question. (iii) **Input Tokens**: The average number of tokens required to submit a request to the LLM. Input tokens generally incur lower costs and have less impact on processing time compared to output tokens. (iv) **Output Tokens**: The average number of tokens generated by the LLM in its response. Output tokens are more critical due to their higher cost and significantly longer processing time compared to input tokens. Optimizing output tokens is crucial for reducing response time and improving LLM performance.

To assess the significance of each component in the chatbot architecture, we conducted six experiments using the same dataset of 298 questions. Experiment 1 employed the full architecture with fine-tuned models of Gliner, BGE Rerank, and LLM-Embedder, while Experiment 2 used the full architecture with raw models. In subsequent experiments, we systematically removed specific modules: Experiment 3 excluded the *Query Refinement* module; Experiment 4 excluded the *Entity Recognition* module; Experiment 5 removed both *Tool Filtering* modules; and Experiment 6 relied solely on LLM prompts without any additional modules.

[7] `https://github.com/Duc-NSH/blockchain-chatbot/blob/main/test_questions.jsonl`

7.2 Experimental Results

The experimental results[8], summarized in Table 1, indicate that the chatbot performs best with the complete architecture, achieving an accuracy rate of 91.95%, demonstrating the effectiveness of the proposed architecture in providing accurate responses in most cases. The average response time was 6.83 seconds, meeting real-time response criteria and ensuring a seamless user experience. In Experiment 2, using the full architecture with raw models, accuracy dropped to 73.49%, highlighting the significant improvement achieved through fine-tuning.

Table 1: Experiment results

Experiment	Accuracy Rate	Response Time	Input Tokens	Output Tokens
E1: Full Architecture (Fine-tuned Models)	91.95% (274/298)	6.83 seconds	2,201 tokens	190 tokens
E2: Full Architecture (Raw Models)	73.49% (219/298)	7.20 seconds	2,209 tokens	199 tokens
E3: Removing Query Refinement Module	44.97% (134/298)	8.75 seconds	3,443 tokens	198 tokens
E4: Removing Entity Recognition Module	76.85% (229/298)	6.29 seconds	2,151 tokens	229 tokens
E5: Removing Tool Filtering Modules	81.21% (242/298)	9.28 seconds	7,636 tokens	283 tokens
E6: LLM with Prompts Only	54.70% (163/298)	5.80 seconds	4,933 tokens	150 tokens

In Experiment 3, removing the *Query Refinement* module caused accuracy to drop sharply to 44.97%. Without refined queries, the system had to process both dialogue history and the current query at each step, leading to increased input tokens. This affected the performance of the lightweight models LLM-Embedder and BGE-Rerank, which only achieve high accuracy with independent queries. Moreover, the excess information in the prompt hindered the agent's ability to select appropriate tools, resulting in a further decline in accuracy.

Similarly, in Experiment 4, removing the *Entity Recognition* reduced accuracy to 76.85%, particularly affecting queries involving entities not previously trained by the LLMs. In Experiment 5, excluding the *Tool Filtering* reduced accuracy to 81.21%, as the absence of filtering led to an explosion in input tokens and increased complexity in tool context usage. This highlights the critical role of tool filtering in maintaining system performance and scalability. In the final experiment, using only LLM prompts resulted in the fastest response time due to the simplified architecture but dropped accuracy significantly to 54.70%. This emphasizes the importance of the architectural components for both performance and cost-efficiency.

We observed that removing modules does not necessarily reduce the chatbot's response time, as each module plays a distinct role. Their absence may cause confusion within the agent system, leading to redundant or suboptimal planning, which can increase response time. This is due to prompts either lacking essential information or being overloaded with irrelevant data, resulting in inefficiencies.

8 Conclusion

This paper presents a novel approach to enhancing chatbot functionality by LLMs. Our proposed system enables users to interact directly with APIs without the need for intermediate programming or application development. We address the limitations of traditional LLM-based chatbots which are constrained by their training data and lack of real-time information access. We have detailed a comprehensive architecture that includes distinct modules for query refinement, entity recognition, and tool filtering, supported by a multi-agent system for planning, validation, and response generation. Our experimental results demonstrate that the system achieves a high accuracy rate of 91.95% with minimal response time. The proposed architecture and methodology not only enhance the usability

[8] `https://github.com/Duc-NSH/blockchain-chatbot/tree/main/test_results`

of blockchain technology but also provide a scalable solution applicable across various domains.

Future work will focus on extending the system's capabilities to additional domains and refining the integration of emerging AI models to further enhance performance and user experience.

References

[1] Akari Asai et al. "Task-aware Retrieval with Instructions". In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 3650–3675.

[2] Jaekeol Choi et al. "Improving Bi-encoder Document Ranking Models with Two Rankers and Multi-teacher Distillation". In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '21. Virtual Event, Canada: Association for Computing Machinery, 2021, pp. 2192–2196.

[3] Marco Del Tredici et al. "Question Rewriting for Open-Domain Conversational QA: Best Practices and Limitations". In: *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. CIKM '21. Virtual Event, Queensland, Australia: Association for Computing Machinery, 2021, pp. 2974–2978.

[4] Wenqi Fan et al. "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '24. Barcelona, Spain: Association for Computing Machinery, 2024, pp. 6491–6501.

[5] GeeksforGeeks. *Agents in Artificial Intelligence*. Last updated: 05 June 2023. 2023.

[6] Zhibin Gou et al. *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*. 2024. arXiv: 2305.11738 [cs.CL].

[7] Ziwei Ji et al. "Survey of Hallucination in Natural Language Generation". In: *ACM Comput. Surv.* 55.12 (Mar. 2023).

[8] kanishk khatter. *Agentic RAG : Unleashing the Power of Agent-Based Tools*. 2024.

[9] Sehoon Kim et al. "An LLM Compiler for Parallel Function Calling". In: *arXiv* (2023).

[10] Nelson F. Liu et al. "Lost in the Middle: How Language Models Use Long Contexts". In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173.

[11] Anqi Mao, Mehryar Mohri, and Yutao Zhong. "Cross-entropy loss functions: theoretical analysis and applications". In: *Proceedings of the 40th International Conference on Machine Learning*. ICML'23. Honolulu, Hawaii, USA: JMLR.org, 2023.

[12] Changle Qu et al. "Tool Learning with Large Language Models: A Survey". In: *ArXiv* abs/2405.17935 (2024).

[13] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI].

[14] Aditi Singh et al. "Enhancing AI Systems with Agentic Workflows Patterns in Large Language Model". In: *2024 IEEE World AI IoT Congress (AIIoT)*. 2024, pp. 527–532.

[15] Hongjin Su et al. "One Embedder, Any Task: Instruction-Finetuned Text Embeddings". In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 1102–1121.

[16] Hongru Wang et al. "Empowering Large Language Models: Tool Learning for Real-World Interaction". In: *Proceedings of the 47th International ACM SIGIR Confer-*

ence on Research and Development in Information Retrieval. SIGIR '24. Washington DC, USA: Association for Computing Machinery, 2024, pp. 2983–2986.

[17] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS '22. New Orleans, LA, USA: Curran Associates Inc., 2024.

[18] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *ArXiv* abs/2210.03629 (2022).

[19] Urchade Zaratiana et al. *GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer*. 2023. arXiv: 2311.08526 [cs.CL].

[20] Peitian Zhang et al. "Retrieve Anything To Augment Large Language Models". In: *CoRR* abs/2310.07554 (2023). arXiv: 2310.07554.